C Practice Test

Imperial College London

Department of Computing

Simple Regular Expressions

Friday 6th June 2025
16h00 to 17h00
ONE HOUR

- This practice test is **unassessed**. However, if it were assessed, the parts would have carried *30%, 30% and 40% of the marks.*
- Attempt all parts. If you cannot get one of the parts to work, try the next one. Each part may be done entirely independently.
- Your code needs to compile and work correctly in the test environment which will be the same as the lab machines. Comment out any code that does not compile before you finish. If this test was assessed, 3% would be deducted from any code that does not compile.
- **Important:** You should only modify the files each part tells you to. All other files should be considered read-only. All files for each part are in a separate **~/re/PartN** directory.
- Feel free to define any of your own helper functions which would help to make your code more elegant; make them static.
- Remember standard header files such as `<string.h>` and `<assert.h>`. Remember how to check man pages (`man strcpy` and so on).
- You may compile your code however you like; we recommend using **CBuild (aka cb)** as usual, and have supplied suitable **.cbuild** files for each part. No assistance will be given with compiling your code any other way than with CB.

1 Introduction - Problem Description

Regular expressions or **regexes**, are used to describe search patterns in text processing applications. Examples of their use include Linux's **grep** filter utility, text editors like **vim** and **emacs** and also programming languages like Perl, Python, Java, C and even Haskell. The basic use of regexes is to determine whether a target string *matches* a regex.

In this test, we are going to explore some of the algorithms and data structures that can be used to implement (simplified) regular expressions.

In particular, we will first build (or complete) some data structures to represent regular expressions, then we will do some work on parsing a regex string into that internal form, and then we will implement a recursive pattern match algorithm that will try to match a regex against a target string, reporting whether or not it matches and – if it does match – which portion of the target string it matches.

2 Simple Regular Expressions

In this test, we will work with a small subset of full-blown regular expressions, which offer a good proportion of the power of full-blown regexes while being much simpler to implement. Specifically:

Single Character Regexes

- A *terminal character* such as `a` matches itself.
- A *range* such as `[abc]` matches any one of those characters, i.e. either an `'a'` or a `'b'` or a `'c'`. That is, it represents a set from which we match any member.

  A range can contain any number of characters, and for convenience you can write `[a-j]` as a shorthand for `[abcdefghij]`. A range may contain a mixture of individual characters and contiguous ranges, such as `[aep-tz]`.
- An *any* (written as `.`) matches *any single non-NUL character* - i.e. it matches any single character **apart from** the `'\0'` at the end of the target string.

Sequencing and anchoring

- A *sequence* of any two or more simple patterns placed next to each other, such as `ab[c-d]` matches each simple pattern in turn against adjacent characters in the target string. In this case, it matches any target string that contains, somewhere within it, an `'a'` immediately followed by a `'b'` immediately followed by either a `'c'` or a `'d'`.

  Note that any string literal regex such as `hello` (as long as it doesn't contain any special regex characters) matches that literal string, somewhere within the target string.
- Normally a regex matches anywhere in the target string. A *start anchor* (written as `^` at the start of a regex) causes the regex to only match against the start of the target string. So for example `^h[ei]` only matches target strings in which the first character is an `'h'` and the second character is either `'e'` or `'i'` – that is, any target string that starts with `he` or `hi`.

- Similarly, an *end anchor* (written as `$` at the end of a regex) causes the regex to only match against the end of the target string. So for example `h[ei]$` only matches target strings in which the last two characters are `'h'` and either `'e'` or `'i'` – that is, any target string ending in `he` or `hi`.

Repetitions

- *Optional: match zero or one time* – any single character pattern may be made *optional*, by appending a `?` to it. So, for example, `a?` matches the empty string or a single `'a'`, `[a-c]?` matches the empty string or a single `'a'`, `'b'` or `'c'`, and `.?` matches the empty string or any single non-empty character (that means it always matches).

- *Kleene Star Repetition: match zero or more times* – any single character pattern may be repeated *zero or more times*, by appending a `*` to it. So, for example, `a*` matches the empty string, or `'a'` or `'aa'`, or `'aaa'`... or any number of adjacent `'a'`s, such as `'aaaaaaaaaaaaaaaaaaaa'`.

  `[a-c]*` matches the empty string, or a single `'a'`, `'b'` or `'c'`, or two adjacent letters each from the set `'a'..'c'` (eg. `bc`), or any number of adjacent letters from the set `'a'..'c'` (eg. `abcaababcaaabbcacb`).

  `.*` matches the empty string or any single character or any two adjacent characters.. or any number of adjacent characters. In other words it matches any amount of text. This is often used as a separator, as in `[a-z].*[0-9]` which matches a letter followed later by a digit.

- *Kleene Plus Repetition: match one or more times* – any single character pattern may be repeated *one or more times*, by appending a `+` to it. So, for example, `a+` matches `'a'` or `'aa'` or `'aaa'`... or any number of adjacent `'a'`s, such as `aaaaaaaaaaaaaaaaaaaa`. But unlike `a*` it doesn't match the empty string.

  `[a-c]+` matches a single `'a'`, `'b'` or `'c'`, or two adjacent letters each independently chosen from the set `'a'..'c'` (eg. `bc`), or any number of adjacent letters from the set `'a'..'c'` (eg. `abcaababcaaabbcacb`). But unlike `[a-c]*` it doesn't match the empty string.

  `.+` matches any single character or any two adjacent characters.. or any number of adjacent characters. In other words it matches any non-empty amount of text. Unlike `.*` it doesn't match the empty string.

Pre-supplied files

You are supplied with a number of files in the `~/re` directory:

```
README:                 readme file that describes the other files
labsearchstr:           pre-compiled working regex string matcher
summarisetests:         C-Tools style test summariser

Part1/README:           readme file that describes Part 1
Part1/regex.[ch]:       abstract regex types - skeleton
Part1/testregex.c:      unit test program for abstract regex types

Part2/README:           readme file that describes Part 2
```

```
Part2/parseregex.[ch]:    Translate the regex string into an abstract
                          regex - skeleton
Part2/testparseregex.c:   Unit test program for regex parsing.

Part3/README:             readme file that describes Part 3
Part3/match.[ch]:         regex string matching and reporting code -
                          skeleton
Part3/testmatch.c:        Unit test program for regex matching.
Part3/searchstr.c:        Match a regex to a target string on the
                          command line.
```

You can use `labsearchstr` to explore your understanding of regex pattern matching against particular target strings. If you'd like to spend a few minutes doing that, some possible invocations are shown in the **README**. For example:

```
./labsearchstr 'hello' "hell helloo there"
```

reports:

```
have matched RE /hello/ against target: hell <hello>o there
```

Specific Tasks

1. Cd into the **Part1/** directory, read the **README** file, and then compile everything up via `cb`. Run the unit test `./testregex` and you will see that it aborts. Why? The file **regex.c** contains a large number of ADT regex functions – but two of those ADT functions are stubs:

   ```
   SPList l = make_SPList( maxsize );
   ```

   ```
   push_SPList( l, simplepattern );
   ```

   Read **regex.h** to discover how the types `SPList` and `SimplePat` etc are defined. Then implement the stubbed functions in **regex.c**, in order to get the unit test to pass all it's tests (once it no longer aborts, you may prefer to run `cb --test` which summarises the test results). When all your unit tests pass, you may also want to run `valgrind ./testregex` to check that your code doesn't leak memory, overrun the ends of arrays, etc.

2. Cd into the **Part2/** directory, read the **README** file, and then compile everything up via `cb`. In the file **parseregex.c** you will find an incomplete function:

   ```
   RE re = parse_regex( flags, str );
   ```

   As a result of this, the unit test program `./testparseregex` (invoke it either directly or via `cb --test`) fails well over half it's tests.

   Currently, `parse_regex()` implements all the simple code, i.e. looking for a '`.`', or a '`[`' and then parsing a range if it sees one. But it has no code to look for an optional repetition character ('`?`', '`*`' or '`+`'), and no code to use the information gathered to construct the correct `SimpleType` (refresh your memory about that type from Part1/regex.h). There is a useful comment in `parse_regex()` to guide you.

   Implement the missing code and get the unit test to pass all it's tests.

3. Next, cd into the **Part3/** directory, read the **README** file, then compile everything up via `cb`. Run the unit test `./testmatch` (either directly or via `cb --test`) and you will see that it fails about half it's tests.

   In **match.c**, the broad structure of recursively pattern matching an abstract regex against a target string has been implemented. In particular, two implemented functions:

   `bool ismatched = match( re, target, &startpos, &endpos );` and

   `bool matched = match_at( currpos, target, nel, l, endanchor, &endpos );`

   handle most of the recursive matching - start and end anchors, and the broad structure of trying to match a list of simple patterns against the target starting at a particular position, by trying to match the head of the pattern list against the first few characters of the target string, and then recursively trying to match the rest of the target against the tail of the pattern list.

   The innermost part of the matching process is a function that matches a simple pattern against the front of the target string, determines whether or not it does match, and determines (if it does match) the **range of how many target characters the head pattern may match** – but this function is incomplete:

   `bool matched = match_simplepat( sp, target, currpos, &firstoff, &noff );`

   it currently handles matching and building match ranges for patterns Dot, One, OptDot and OptOne – but has stubs for DotStar, Star, DotPlus and Plus.

   Implement the missing code – once you get those 4 missing cases correct, the entire unit test should pass all it's tests.

   Note that the range of offsets is represented by **(firstoff, noff)**, the first offset in the range and the number of offsets in the range. For example, suppose the pattern (`sp`) is `Star("ab")`, and the target string (`target`) is `"abracadabra"`. `Star("ab")` can match zero characters from the target, or 1 character (the leading `'a'`) or 2 characters (the leading `'ab'`). So this range would be represented by **firstoff=0, noff=3** – you might prefer to think of this range as **firstoff=0..lastoff=noff-1=2**.

   Note that there is a commented out `assert` at the end of each stubbed case. Each `assert` checks the range post-condition for that case – if the case matches. You may wish to uncomment it and move it into your code when you know the case has successfully matched, in order to help you debug your code.

4. Bonus task: if you have a few minutes left at the end, and have got all of questions 1-3 working, you might like to try assembling all your solutions together and see if they give you a working regex pattern matching system – just for the satisfaction.

   Go into the `BONUS` directory, and run `./copyin`. That should copy in your solutions from `../Part*`. Then you should be able to run `cb` to compile all your code together. If all goes well, the unit tests and the final program `searchstr` should work.

   Invoke `searchstr` just like the top-level README described running `labsearchstr`. In fact, why not run both `labsearchstr` and `searchstr` on each test regex and target string, and compare the results.

Good luck!